

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №33

Тема: Створення алгоритмів

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Закріпити вміння самостійно розробляти алгоритми: за постановкою задачі обирати підхід (повний перебір, сортування, двійковий пошук, два вказівники, жадібний алгоритм, динамічне програмування, хешування), обґрунтовувати його коректність і складність, реалізовувати та перевіряти експериментально, порівнюючи наївний і ефективний розв'язки.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal), бібліотека matplotlib (для графіків).

Короткі теоретичні відомості

1. Алгоритм розв'язання нової задачі. Для задачі, яку не було розглянуто раніше, використовують послідовність дій:

- Зрозуміти постановку:** вхідні й вихідні дані, обмеження на розміри; виписати 2–3 малі приклади й розв'язати їх вручну.
- Розробити наївний розв'язок** (зазвичай повний перебір): він простий і слугує еталоном для перевірки.
- Оцінити його складність:** чи вкладеться він у обмеження за розмірами? (За 1 с Python виконує порядку 10^7 простих операцій.)
- Знайти вузьке місце** і застосувати відповідну ідею оптимізації (таблиця нижче).
- Перевірити коректність:** порівняти ефективний розв'язок із наївним на великій кількості випадкових малих тестів.
- Виміряти час** на великих даних і підтвердити очікувану асимптотику.

2. Типові ідеї оптимізації:

Ознака в задачі	Ідея	Приклад ефекту
Багаторазовий пошук елемента	хеш-таблиця (<code>set</code> , <code>dict</code>) — пошук за $O(1)$	$O(n^2) \rightarrow O(n)$
Дані впорядковані або їх можна впорядкувати	двійковий пошук; два вказівники	$O(n^2) \rightarrow O(n \log n)$ або $O(n)$
Повторні обчислення одних і тих самих підзадач	динамічне програмування (memoізація / табуляція)	$O(2^n) \rightarrow O(n^2)$ або $O(n)$
Потрібні суми / кількості на відрізках	префіксні суми: $\text{pref}[i] = a[0] + \dots + a[i-1]$; сума на $[l, r) = \text{pref}[r] - \text{pref}[l]$	$O(n)$ на запит $\rightarrow O(1)$
Локально оптимальний вибір безпечний	жадібний алгоритм	$O(2^n) \rightarrow O(n \log n)$

Ознака в задачі	Ідея	Приклад ефекту
Максимум / мінімум на неперервному відрізку	алгоритм Кадане (ДП із одним числом стану)	$O(n^2) \rightarrow O(n)$

3. Метод двох вказівників. Два індекси рухаються по масиву (з кінців назустріч або в один бік), так що кожен елемент розглядається не більше ніж один раз; разом дає $O(n)$. Класичний приклад — знайти в **впорядкованому** масиві пару чисел із заданою сумою: якщо сума занадто мала — збільшуємо лівий вказівник, якщо зavelика — зменшуємо правий.

4. Алгоритм Кадане. Максимальна сума неперервного підмасиву: проходимо масив, підтримуючи `cur` — найбільшу суму підмасиву, що закінчується в поточній позиції: `cur = max(a[i], cur + a[i])`, `best = max(best, cur)`. Це динамічне програмування зі станом із одного числа; складність $O(n)$.

5. Тестування. Окрім ручних прикладів, використовують **стрес-тестування**: генерують багато випадкових малих вхідних наборів і порівнюють результат ефективного алгоритму з наївним; розбіжність відразу виявляє помилку та дає мінімальний контрприклад. Обов'язково перевіряють граничні випадки (порожній масив, один елемент, усі елементи однакові, від'ємні числа).

Хід роботи

Завдання 1. Вибір методу за постановкою задачі

Для кожної задачі визначити тип розв'язку, оцінити складність наївного та ефективного підходів та вказати практичні роботи, матеріал яких застосовується.

- 1) Знайти в масиві з $n = 10^5$ чисел дві, сума яких дорівнює заданому числу S .
- 2) Дано $n = 10^5$ чисел (можуть бути від'ємними). Знайти найбільшу суму неперервного підмасиву.
- 3) Є $q = 10^5$ запитів «сума елементів масиву з індексу l по r », $n = 10^5$.
- 4) Є n занять із часом початку й кінця. Провести якнайбільше занять без перетинів.
- 5) Знайти найдовшу спільну підпослідовність двох рядків довжиною до 2000.
- 6) Перевірити, чи є у списку з $n = 10^6$ чисел дублікати.
- 7) Розмінити суму 10^4 монетами довільних номіналів із мінімальною кількістю монет.
- 8) Знайти всі пари (a, b) натуральних чисел до 1000 із $a^2 + b^2 = c^2$ для натурального c .

Порядок виконання:

1. Для кожної задачі запишіть наївний розв'язок (одним реченням) та оцініть кількість операцій для вказаних розмірів (наприклад, $n^2 = 10^{10}$ — зavelика).
2. Запишіть ефективну ідею з таблиці теоретичної частини (хешування, два вказівники, префіксні суми, Кадане, жадібність, ДП) та її складність.
3. Складіть підсумкову таблицю «№ — наївний (складність) — ефективний (ідея, складність) — практична робота».
4. Для задач 1 і 4 обґрунтуйте, чому обрана ідея дає правильну відповідь (для 4 — спиріться на практичну роботу №30).
5. Для задач, де є кілька підходів (1, 6), порівняйте їх за часом і додатковою пам'яттю.

Завдання 2. Ручна трасування ефективних алгоритмів

Виконати вручну алгоритм Кадане та метод двох вказівників на малих прикладах і порівняти з наївним розв'язком.

Порядок виконання:

1. Для масиву $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ виконайте алгоритм Кадане: складіть таблицю « $i - a[i] - cur - best$ »; запишіть найбільшу суму та відповідний підмасив.
2. Знайдіть ту саму суму наївно (усі підмасиви) для масиву $[3, -2, 4]$ – перелічіть усі 6 підмасивів і їх суми; порівняйте з результатом Кадане.
3. У впорядкованому масиві $[1, 3, 4, 6, 8, 11, 15]$ знайдіть пару з сумою 14 методом двох вказівників: складіть таблицю « $left - right - a[left] + a[right] - рішення$ ».
4. Побудуйте масив префіксних сум для $[5, 2, 7, 1, 4, 3]$ і знайдіть суму елементів з індексами $2 \dots 4$ за формулою $pref[r+1] - pref[l]$.
5. Порівняйте кількість операцій наївних і ефективних методів для $n = 10^5$.

Завдання 3. Програмна реалізація: наївні й ефективні розв'язки та стрес-тест

Реалізувати наївні й ефективні алгоритми для трьох задач і перевірити їх стрес-тестуванням.

Порядок виконання:

1. Задача «Пара із заданою сумою»: реалізуйте наївний варіант `two_sum_naive(a, S)` (два вкладені цикли), варіант із множиною `two_sum_hash(a, S)` та варіант двох вказівників для впорядкованого масиву `two_sum_two_pointers(a, S)`; функції повертають пару значень або None.
2. Задача «Максимальний підмасив»: реалізуйте `max_subarray_naive(a)` за $O(n^2)$ (перебір початку й кінця з накопиченням суми) та `max_subarray_kadane(a)` за $O(n)$.
3. Задача «Суми на відрізках»: реалізуйте `range_sum_naive(a, l, r)` (сума циклом) та клас-обгортку із префіксними сумами (функції `build_prefix(a)` та `range_sum(pref, l, r)`).
4. Напишіть стрес-тест: для 500 випадкових масивів малих розмірів (довжина $1 \dots 8$, значення $-10 \dots 10$) порівняйте результати наївного й ефективних методів (для пари із заданою сумою порівнюйте лише факт існування пари).
5. Перевірте граничні випадки: масив із однієї від'ємної величини, усі елементи від'ємні, порожній масив для суми пари.

Приклад реалізації мовою Python:

python

```
import random

def two_sum_naive(a, S):
    for i in range(len(a)):
        for j in range(i + 1, len(a)):
            if a[i] + a[j] == S:
                return a[i], a[j]
    return None

def two_sum_hash(a, S):
    seen = set()
    for x in a:
        if S - x in seen:
            return S - x, x
        seen.add(x)
    return None
```

```

def two_sum_two_pointers(sorted_a, S):
    left, right = 0, len(sorted_a) - 1
    while left < right:
        s = sorted_a[left] + sorted_a[right]
        if s == S:
            return sorted_a[left], sorted_a[right]
        if s < S:
            left += 1
        else:
            right -= 1
    return None

def max_subarray_naive(a):
    best = a[0]
    for i in range(len(a)):
        s = 0
        for j in range(i, len(a)):
            s += a[j]
            best = max(best, s)
    return best

def max_subarray_kadane(a):
    cur = best = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

def build_prefix(a):
    pref = [0]
    for x in a:
        pref.append(pref[-1] + x)
    return pref

def range_sum(pref, l, r):
    return pref[r + 1] - pref[l]

print("Кадане [-2,1,-3,4,-1,2,1,-5,4]:", max_subarray_kadane([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
print("Усі від'ємні [-8, -3, -6]:", max_subarray_kadane([-8, -3, -6]),
max_subarray_naive([-8, -3, -6]))
print("Два вказівники [1,3,4,6,8,11,15], S = 14:", two_sum_two_pointers([1, 3, 4, 6, 8, 11, 15], 14))
pref = build_prefix([5, 2, 7, 1, 4, 3])
print("Префіксні суми:", pref, "; сума [2..4] =", range_sum(pref, 2, 4))
print("Порожній масив:", two_sum_naive([], 5), two_sum_hash([], 5))

random.seed(11)
bad = 0
for _ in range(500):
    a = [random.randint(-10, 10) for _ in range(random.randint(1, 8))]
    S = random.randint(-10, 10)
    if max_subarray_naive(a) != max_subarray_kadane(a):
        bad += 1
    exists = two_sum_naive(a, S) is not None
    if exists != (two_sum_hash(a, S) is not None) or exists !=
(two_sum_two_pointers(sorted(a), S) is not None):
        bad += 1
    l = random.randint(0, len(a) - 1)
    r = random.randint(l, len(a) - 1)
    if sum(a[l:r + 1]) != range_sum(build_prefix(a), l, r):
        bad += 1

```

```
print("Стрес-тест із 500 випадкових наборів: розбіжностей =", bad)
```

Результат виконання програми:

```
Кадане [-2,1,-3,4,-1,2,1,-5,4]: 6
Усі від'ємні [-8, -3, -6]: -3 -3
Два вказівники [1,3,4,6,8,11,15], S = 14: (3, 11)
Префіксні суми: [0, 5, 7, 14, 15, 19, 22] ; сума [2..4] = 12
Порожній масив: None None
Стрес-тест із 500 випадкових наборів: розбіжностей = 0
```

Висновок за результатами: ефективні алгоритми дають ті самі відповіді, що й наївні: у стрес-тесті з 500 випадкових наборів розбіжностей не знайдено, а результати ручних трасувань (найбільша сума 6, пара (3, 11) із сумою 14, сума елементів 2...4 дорівнює 12) збігаються з відповідями програми. Алгоритм Кадане коректно працює і для масиву з усіх від'ємних чисел (відповідь -3 - найбільший елемент), якщо ініціалізувати *best* першим елементом, а не нулем: початкове значення 0 дало б неправильну відповідь «порожній підмасив».

Завдання 4. Експериментальне порівняння складності

Виміряти час роботи наївних і ефективних алгоритмів для зростаючих n , побудувати графік і підтвердити асимптотику.

Порядок виконання:

1. Для $n = 500, 1000, 2000, 4000$ згенеруйте випадкові масиви (значення $-10^6 \dots 10^6$, випадкова S) і виміряйте час (найкращий із 3 запусків) для: `two_sum_naive` та `two_sum_hash` (щоб гарантовано не знайти пари, задайте S більшим за максимально можливу суму), `max_subarray_naive` та `max_subarray_kadane`.
2. Занесіть результати в таблицю « n — naive two_sum — hash — naive subarray — Kadane».
3. Для кожного алгоритму обчисліть відношення часів при подвоєнні n ; підтвердіть, що для наївних методів воно близьке до 4 ($O(n^2)$), а для ефективних — до 2 ($O(n)$).
4. Для $q = 20\,000$ запитів на масиві $n = 20\,000$ порівняйте час наївних сум на відрізках та сум із префіксами; підрахуйте, у скільки разів префіксні суми швидші.
5. Побудуйте графік залежності часу від n для обох задач (наївний і ефективний методи) та сформулюйте висновок.

Приклад реалізації мовою Python:

```
python
import time
import matplotlib.pyplot as plt

random.seed(1)

def best_time(func, *args, repeats=3):
    best = float("inf")
    for _ in range(repeats):
        t0 = time.perf_counter()
        func(*args)
        best = min(best, time.perf_counter() - t0)
    return best
```

```

sizes = [500, 1000, 2000, 4000]
rows = {"two_sum_naive": [], "two_sum_hash": [], "subarray_naive": [], "kadane": []}
print("      n      two_sum naive      hash      subarray naive      Kadane")
for n in sizes:
    a = [random.randint(-10**6, 10**6) for _ in range(n)]
    S = 10**7
    t = [best_time(two_sum_naive, a, S), best_time(two_sum_hash, a, S),
          best_time(max_subarray_naive, a), best_time(max_subarray_kadane, a)]
    for key, v in zip(rows, t):
        rows[key].append(v)
    print(f"{n:>6}      {t[0]:>11.5f}      {t[1]:>8.6f}      {t[2]:>12.5f}      {t[3]:>8.6f}")

print("\nВідношення часу при подвоєнні n (naive two_sum / hash / naive subarray /
Kadane):")
for i in range(1, len(sizes)):
    r = [rows[k][i] / rows[k][i - 1] for k in rows]
    print(f"      {sizes[i - 1]:>5} -> {sizes[i]:>5}: " + " / ".join(f"x{v:.1f}" for v in
r))

n = q = 20000
arr = [random.randint(-100, 100) for _ in range(n)]
queries = []
for _ in range(q):
    l = random.randint(0, n - 1)
    queries.append((l, random.randint(l, n - 1)))
t_naive = best_time(lambda: [sum(arr[l:r + 1]) for l, r in queries], repeats=1)
pref = build_prefix(arr)
t_pref = best_time(lambda: [range_sum(pref, l, r) for l, r in queries])
print(f"\nЗапити сум (n = q = 20000): наївно {t_naive:.3f} с, префіксні суми
{t_pref:.4f} с, прискорення x{t_naive / t_pref:.0f}")

plt.figure(figsize=(7, 4.2))
plt.plot(sizes, rows["two_sum_naive"], "o-", label="пара із сумою: наївний O(n²)")
plt.plot(sizes, rows["two_sum_hash"], "o-", label="пара із сумою: хеш O(n)")
plt.plot(sizes, rows["subarray_naive"], "s--", label="підмасив: наївний O(n²)")
plt.plot(sizes, rows["kadane"], "s--", label="підмасив: Кадане O(n)")
plt.xlabel("розмір масиву n")
plt.ylabel("час, с")
plt.title("Наївні та ефективні алгоритми")
plt.grid(alpha=0.4)
plt.legend()
plt.show()

```

Результат виконання програми:

n	two_sum naive	hash	subarray naive	Kadane
500	0.00530	0.000030	0.01526	0.000096
1000	0.02159	0.000063	0.05977	0.000186
2000	0.08658	0.000159	0.24006	0.000395
4000	0.34835	0.000295	1.02299	0.000784

Відношення часу при подвоєнні n (naive two_sum / hash / naive subarray / Kadane):

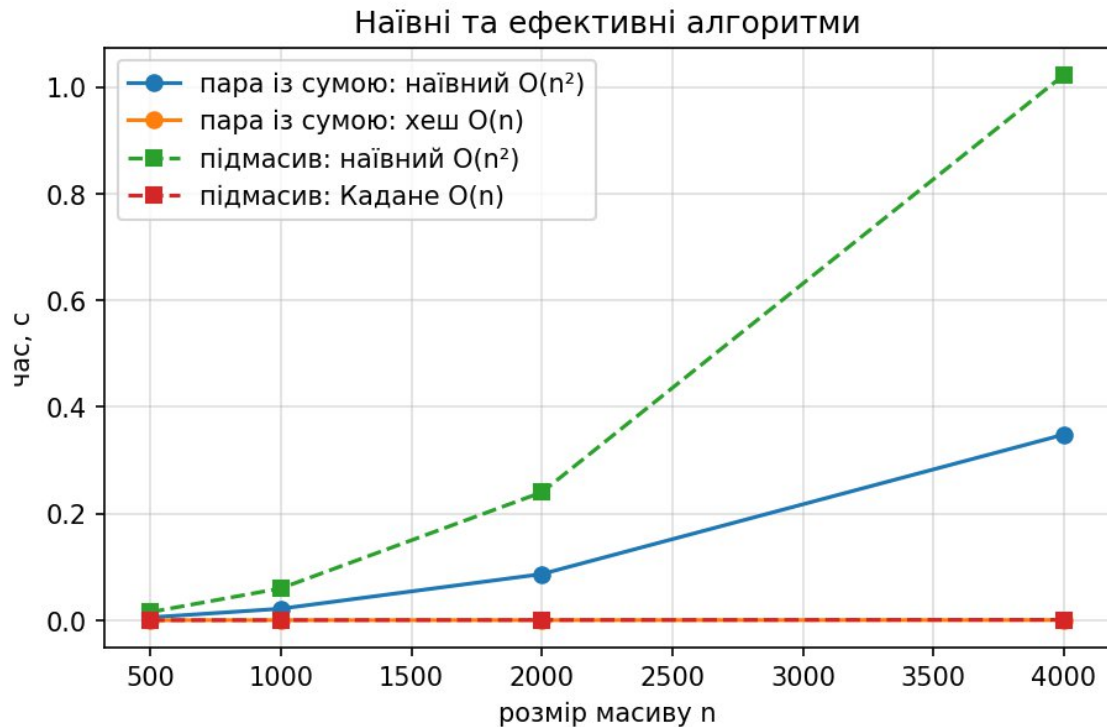
```

500 -> 1000: x4.1 / x2.1 / x3.9 / x1.9
1000 -> 2000: x4.0 / x2.5 / x4.0 / x2.1
2000 -> 4000: x4.0 / x1.9 / x4.3 / x2.0

```

Запити сум (n = q = 20000): наївно 2.352 с, префіксні суми 0.0018 с, прискорення x1340

Графік порівняння:



Висновок за результатами: для наївних методів час зростає приблизно у 4 рази при подвоєнні n ($O(n^2)$), а для хешування та алгоритму Кадане — приблизно у 2 рази ($O(n)$). Уже при $n = 4000$ різниця становить кілька порядків: ефективні методи працюють приблизно за 0,3–0,8 мс, наївні — за 0,3–1 с, і розрив зростає із зростанням n (абсолютні значення залежать від комп'ютера). Префіксні суми дозволяють відповідати на запити за $O(1)$ після лінійної підготовки, тому при великій кількості запитів виграш сягає порядку тисяч разів. Ефективні алгоритми потрібно перевіряти стрес-тестом проти наївних, щоб оптимізація не порушила коректності.

Контрольні питання

1. Як діяти, якщо задача нова й готового алгоритму немає? З яких кроків складається розроблення?
2. Чому корисно починати з наївного розв'язку, і як він допомагає при тестуванні?
3. Скільки простих операцій за секунду може виконати програма на Python? Як це використовується при оцінці складності?
4. Опишіть метод двох вказівників. За яких умов його можна застосувати до задачі пошуку пари із заданою сумою?
5. У чому ідея алгоритму Кадане? Чому для масиву з усіх від'ємних чисел його треба ініціалізувати першим елементом?
6. Що таке префіксні суми та яку складність вони дають для запитів суми на відрізку?
7. Що таке стрес-тестування? Яку інформацію про алгоритм воно дає?
8. Як за експериментом підтвердити, що час роботи алгоритму має складність $O(n)$ чи $O(n^2)$?

Зміст звіту

1. Тема, мета роботи.
2. Таблиця вибору методу (завдання 1) та ручні трасування (завдання 2).

3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Таблиця вимірів часу й графік складності.
5. Висновки.